

Week 3 Lab: Signal Conditioning — ADC Settling & RC Filtering

EE0827 Embedded System Applications

Overview

Build and test circuits that interface real-world signals with your MCU's ADC. You will design voltage dividers, observe how source impedance affects ADC accuracy, and add RC filtering — skills essential for every embedded project.

Time allocation: 90 minutes

- ADC loading and settling (DC + channel switch): 35 min
- RC low-pass filter: 30 min
- Analysis and documentation: 15 min
- Cleanup: 10 min

Learning Objectives

By the end of this lab, you will be able to:

1. Build voltage dividers and verify their ratio
2. Distinguish ADC dynamic settling from static resistive loading
3. Observe RC filter effect on a time-domain waveform
4. Read ADC values through a conditioned input

Safety Gate

Safety Checklist

- ☐ Bench supply current limit set to 100 mA
- ☐ Always measure voltage with multimeter BEFORE connecting to MCU
- ☐ Double-check polarity on all connections
- ☐ MCU powered separately (USB) during initial testing

Protecting Your MCU

1. **Test divider circuits without MCU first** — verify V_{out} is below 3.3V
2. **Measure before connecting** — use multimeter to confirm
3. **Current limit your supply** — protects against wiring mistakes

Required Parts and Tools

Hardware

- STM32 Nucleo-F401RE board
- Bench power supply (5V, current limiting)
- Function generator (for Part 2)
- Multimeter
- Resistors: 1k Ω ($\times 2$), 2k Ω ($\times 1$), 100k Ω ($\times 1$), 200k Ω ($\times 1$)
- Capacitors: 10 μF ($\times 1$)
- Breadboard and jumper wires

Software

- STM32CubeIDE with a CubeMX project configured for:
 - **ADC1**, Channel 0 (PA0), 12-bit resolution, Software trigger
 - **USART2**, 115200 baud, 8N1 (ST-LINK VCP on PA2)
- Code snippets: `adc_conditioned_input.c` (Part 1 DC), `adc_settling_test.c` (Part 1 channel-switch), `rc_filter_capture.c` (Part 2)
- Python 3 with `pyserial` and `matplotlib` (`pip install --user pyserial matplotlib`)

! PRE-LAB SECTION

Complete the following **BEFORE** arriving at the lab session.

Pre-Lab Checklist

Complete before arriving at the lab:

- ☐ Review voltage divider formula and Thévenin output impedance
- ☐ Understand ADC sampling capacitor charging (dynamic loading vs static loading)
- ☐ Understand RC filter frequency response

Pre-Lab Questions

1. The STM32F401 ADC has an internal sampling capacitor $C_{\text{ADC}} = 4\text{pF}$. Why does the source impedance (R_{AIN}) matter even though the ADC's DC input impedance is extremely high? _____

2. How does an RC filter's cutoff frequency change if you double the capacitance? _____

Pre-Lab Calculations

Complete these calculations before lab:

1. Voltage Divider Design

Design a divider to scale 0-5V input to 0-3.3V output.

Required ratio:

$$k = \frac{3.3V}{5V} = \underline{\hspace{2cm}}$$

If $R_2 = 2\text{k}\Omega$, calculate R_1 :

$$R_1 = R_2 \times \left(\frac{1}{k} - 1 \right) = \underline{\hspace{2cm}} \Omega$$

2. Source Impedance (R_{AIN}) for Each Divider

The ADC sees the divider's Thévenin resistance at its input. Calculate $R_{AIN} = R1 \parallel R2$ for both dividers:

Low-Z divider ($R1 = 1k\Omega$, $R2 = 2k\Omega$):

$$R_{AIN} = \frac{1k \times 2k}{1k + 2k} = \text{_____} \Omega$$

High-Z divider ($R1 = 200k\Omega$, $R2 = 100k\Omega$):

$$R_{AIN} = \frac{200k \times 100k}{200k + 100k} = \text{_____} \Omega$$

3. RC Filter Cutoff

For the low-Z divider ($R1 = 1k\Omega$, $R2 = 2k\Omega$) with $C = 10\mu F$ across $R2$, the capacitor sees the Thévenin resistance of the divider.

Thévenin resistance:

$$R_{th} = R1 \parallel R2 = \frac{1k \times 2k}{1k + 2k} = \text{_____} \Omega$$

Cutoff frequency:

$$f_c = \frac{1}{2\pi \times R_{th} \times C} = \text{_____} Hz$$

i IN-LAB SECTION

The following sections are completed during the lab session.

Part 1

ADC Loading & Settling Experiment

35 minutes

In this experiment you build two voltage dividers with the **same ratio** but different impedances, then compare multimeter vs ADC readings at different sampling times. The goal is to understand *why* source impedance matters for ADC accuracy.

Step 1.1: Low-Impedance Divider ($R_{AIN} = 667, \Omega$)

Build this circuit on your breadboard using $R_1 = 1, k\Omega$ and $R_2 = 2, k\Omega$:

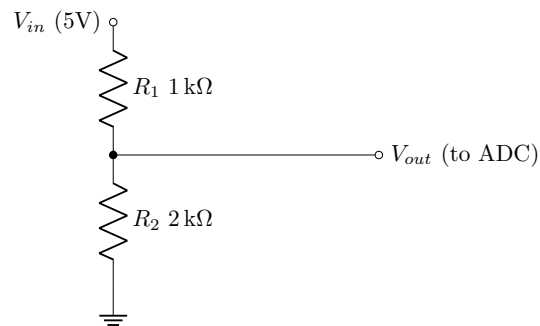


Figure 1: Voltage Divider Circuit

Do NOT connect to MCU yet!

1. Set bench supply to exactly 5.00,V
2. Measure V_{out} with multimeter → expect ~3.33,V

Now connect V_{out} to **PA0** (ADC1_IN0) and run the `adc_conditioned_input` program (see `adc_conditioned_input.c`).

3. In CubeMX, set ADC1 sampling time to **480 cycles** → generate code, build, flash → record ADC reading
4. Change sampling time to **3 cycles** in CubeMX → generate, build, flash → record ADC reading

Condition	Expected V_{out}	ADC Reading	V_{ADC} (mV)
Multimeter	3.33,V	—	—
ADC, 480 cycles	3.33,V	_____	_____
ADC, 3 cycles	3.33,V	_____	_____

Both ADC readings should be close to 3.33,V. The low source impedance ($R_{\text{AIN}} = 667,\Omega$) allows C_{ADC} to charge fully even in the shortest sampling window.

Step 1.2: High-Impedance Divider — DC Input ($R_{\text{AIN}} = 66.7,\text{k}\Omega$)

Build a second divider with the **same ratio** but higher impedance: $R1 = 200,\text{k}\Omega$ and $R2 = 100,\text{k}\Omega$. Same topology as above, different values.

Do NOT connect to MCU yet!

1. Measure V_{out} with multimeter → should still read ~3.33,V (same ratio)
2. Connect V_{out} to **PA0** and run the `adc_conditioned_input` program
3. In CubeMX, set ADC1 sampling time to **480 cycles** → generate, build, flash → record ADC reading
4. Change sampling time to **3 cycles** in CubeMX → generate, build, flash → record ADC reading

Condition	Expected V_{out}	ADC Reading	V_{ADC} (mV)
Multimeter	3.33,V	—	—
ADC, 480 cycles	3.33,V	_____	_____
ADC, 3 cycles	3.33,V	_____	_____

Why Both Readings Look Correct

You will likely see **no error** — even at 3 cycles with $66.7\text{ k}\Omega$ source impedance. This is **expected**.

The ADC's internal sampling capacitor (C_{ADC}) retains its charge between conversions. When you read the same DC voltage on the same channel repeatedly, C_{ADC} is already at the correct voltage from the previous sample. It barely needs to charge, so even 3 cycles is enough.

The settling error only appears when there is a large voltage change between consecutive samples. The next step demonstrates this.

Step 1.3: Channel-Switching Settling Test

The settling error appears when C_{ADC} must swing a large voltage in one sampling window. The real-world trigger for this is **multi-channel scanning**: the ADC reads one channel, then immediately reads another at a very different voltage.

To force this worst case, connect **PA1** to **GND** with a short jumper. The test firmware (`adc_settling_test.c`) reads PA1 (0,V) before every PA0 read, draining C_{ADC} to 0,V each time. Then the PA0 read must charge C_{ADC} from 0,V to V_{divider} through R_{AIN} in a single sampling window.

CubeMX change: Add **ADC1 Channel 1 (PA1)** alongside Channel 0. No scan mode needed — the code selects channels manually.

1. Keep the high-Z divider ($R1 = 200\text{k}\Omega$, $R2 = 100\text{k}\Omega$) with DC 5,V input
2. Connect **PA1** → **GND** with a short jumper wire
3. Flash `adc_settling_test` and run the plotting script:

```
python3 plot_adc_settling.py /dev/ttyACM0    # macOS/Linux
python  plot_adc_settling.py COM3           # Windows
```

You should see:

- **480-cycle trace (blue):** reads ~3,300,mV (correct — enough time to settle)
- **3-cycle trace (red):** reads significantly lower — C_{ADC} cannot charge from 0,V to 3.3,V through $66.7\text{k}\Omega$ in 143,ns

Save or screenshot the dual-trace plot (press Space to pause, Ctrl+C to stop and save CSV).

Now swap in the **low-Z divider** ($R1 = 1\text{k}\Omega$, $R2 = 2\text{k}\Omega$) and repeat. Both traces should overlap — low source impedance lets C_{ADC} settle even in 3 cycles.

Step 1.4: Compare Results

Fill in this comparison table:

	DC single-ch: 3c	DC single-ch: 480c	After GND ch: 3c	After GND ch: 480c
Low-Z (1k / 2k)	_____ mV	_____ mV	_____ mV	_____ mV
High-Z (200k / 100k)	_____ mV	_____ mV	_____ mV	_____ mV

When Does Settling Error Matter?

- **Single channel, DC:** C_{ADC} retains charge between reads — 3 cycles is fine even with high R_{AIN} . You saw this in Step 1.2.
- **After channel switch:** C_{ADC} is at the *previous* channel's voltage. It must charge through R_{AIN} to the new voltage in one sampling window. With high R_{AIN} and 3 cycles, it can't settle → reads low.
- The same effect occurs with **fast-changing signals** (e.g. square waves), but only on the samples immediately after each transition.

Practical rule: Settling error matters when scanning multiple channels or sampling fast-changing signals through high-impedance sources. For a single slow-varying channel, even short sampling times work.

Fix: Lower R_{AIN} (use lower-value divider resistors) or increase sampling time for high-impedance channels.

Checkpoint 1: ADC Settling Understood

Before proceeding to RC filter testing, verify:

- ☐ Both dividers read $\sim 3.33\text{ V}$ with single-channel DC at both sampling times
- ☐ Channel-switch test shows clear error on high-Z divider at 3 cycles
- ☐ Channel-switch test shows no error on low-Z divider
- ☐ You can explain *why* single-channel DC hides the error

Part 2

RC Low-Pass Filter

30 minutes

You will use a function generator, the MCU's ADC, and a Python script to visualize what an RC filter does in real time — no oscilloscope needed.

Step 2.1: Wire Signal Generator Through Divider

Keep the low-Z divider from Part 1 ($R_1 = 1\text{k}\Omega$, $R_2 = 2\text{k}\Omega$) on the breadboard.

- Connect **function generator** output to the divider input (R_1)
- Set to: **square wave, 100,Hz, 3.3,V amplitude, 1.65,V DC offset** (signal swings 0–3.3,V)
- Keep V_{out} connected to **PA0** (ADC input)
- **No capacitor yet**

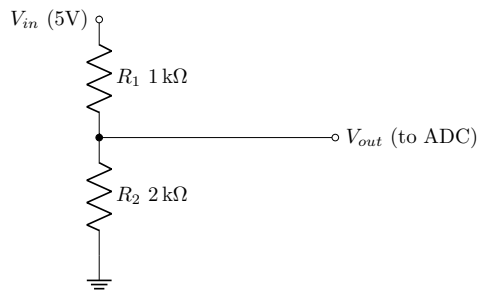


Figure 2: Divider circuit — same topology

The divider scales the signal to 0–2.2,V at V_{out} .

Step 2.2: Capture Unfiltered Signal

Flash the `rc_filter_capture` project (see `rc_filter_capture.c`). This program continuously reads ADC samples and streams them over UART.

Install the Python dependencies (one time):

```
pip install --user pyserial matplotlib
```

If `pip` is blocked on your account, try `pip install --user` or ask your instructor.

Run the live plotting script:

- **Windows:** Find the COM port in Device Manager → Ports (e.g. COM3)

```
python plot_adc.py COM3
```

- **macOS / Linux:**

```
python3 plot_adc.py /dev/ttyACM0
```

The script shows a scrolling time window that updates in real time. You should see a **clean square wave** alternating between $\sim 0\text{mV}$ and $\sim 2,200\text{mV}$.

Save or screenshot this plot (press **Ctrl+C** to stop — the last plot stays on screen).

Step 2.3: Add Filter Capacitor and Compare

Add a **10,μF** capacitor across R_2 :

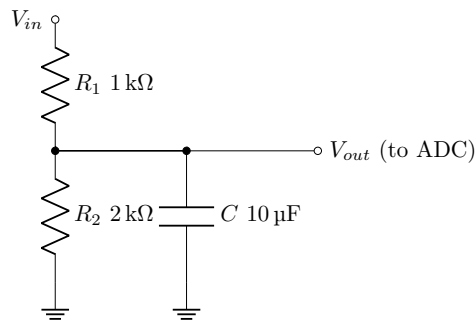


Figure 3: Divider with RC filter

The filter's cutoff frequency (from your pre-lab):

$$f_c = \frac{1}{2\pi \times (R1 \parallel R2) \times C} = \frac{1}{2\pi \times 667 \times 10\mu} \approx 24 \text{ Hz}$$

The 100,Hz square wave is well above f_c — the filter should smooth it heavily.

The live plot will update immediately — you should see the waveform change as the filter takes effect. Compare the two plots:

	Without C	With 10,μF
Waveform shape	_____	_____
Peak voltage (mV)	_____	_____
Trough voltage (mV)	_____	_____

Bonus: Switch the function generator to a **sine wave** at 100,Hz and capture both with and without the capacitor. How does the filtered sine wave differ from the filtered square wave?

Take screenshot: Both plots (unfiltered and filtered) for your lab report.

Checkpoint 2: Filter Effect Visible

Before finishing, verify:

- ☐ Unfiltered plot shows a clean square wave
- ☐ Filtered plot shows a smoothed waveform (rounded transitions or small ripple)
- ☐ You can explain why the capacitor removes the sharp edges

Measurement Summary

ID	Measurement	Expected	Measured	Pass
M1	Low-Z divider ADC @ 3 cycles (DC)	~3.33V (< 2% error)		
M2	High-Z divider ADC @ 3 cycles (DC, single-ch)	~3.33V (C_{ADC} already settled)		
M2b	High-Z divider, after GND channel, 3c vs 480c	3c reads significantly lower		
M3	Unfiltered square wave peak	~2,200 mV		
M4	Filtered (10 μ F) waveform smoothed	Visible in plot		

Failure Injection Test

Demonstrate understanding by predicting failure modes:

Scenario	What Happens?	How to Detect/Prevent?
High-Z divider, 3-cycle, single-channel DC		
High-Z divider, 3-cycle, after GND channel switch		
Filter capacitor shorted		
Divider resistors swapped (R1 R2)		
Ground not connected to MCU ground		

Analysis Questions

Answer these during the lab (2-3 sentences each):

1. The high-Z divider reads correctly at 3 cycles with single-channel DC, but shows error after a channel switch. Explain why in terms of C_{ADC} charging. —
2. A colleague says “the ADC loads the divider like a resistor.” Is this correct? Explain, and describe when the error actually appears. _____

3. Why does the RC filter smooth the square wave? What would happen if you used a 1,μF capacitor instead of 10,μF? _____

Completion Checklist

Before leaving the lab, verify you have accomplished:

- ☐ Low-Z and high-Z divider DC measurements (multimeter + ADC at two sampling times)
- ☐ Channel-switch settling test: dual-trace plot (3c vs 480c) for high-Z divider
- ☐ Comparison table filled in (Step 1.4)
- ☐ Python plots: unfiltered and filtered ADC captures
- ☐ Screenshots: channel-switch dual-trace + unfiltered/filtered plots
- ☐ Answered analysis questions

Success Criteria

This lab is ungraded practice. Use these criteria to gauge your progress:

Area	What to Aim For
ADC settling	DC single-channel reads correctly; channel-switch reveals settling error on high-Z divider
RC filter	Filtering visible in ADC plot
Analysis questions	Demonstrates understanding
Lab technique	Clean measurements, safe practices

Common Issues and Solutions

Issue	Symptom	Solution
Wrong divider ratio	Vout too high/low	Check resistor values, verify connections
ADC reads max (4095)	Vout > 3.3V	Verify divider, check for shorts
Filter not filtering	Plot looks the same	Capacitor not connected, or wrong pin
GPIO damaged	Pin doesn't respond	Replace Nucleo (always test before connecting!)
Noisy ADC readings	Values jumping	Add decoupling cap, check ground

Cleanup Checklist

Before leaving:

- ☐ Turn off bench power supply
- ☐ Disconnect all circuits from MCU
- ☐ Return function generator (if used)
- ☐ Save all Python plots and screenshots
- ☐ Clean up breadboard

Key Takeaways

1. **ADC loading is dynamic, not static** - keep R_{AIN} low or increase sampling time to let C_{ADC} settle
2. **RC filters set maximum signal bandwidth** - choose cutoff carefully
3. **Measure before connecting to MCU** - verify every voltage first